Optimización de Cartera P2P en PrestaMatch – Informe Final
Integrantes: Ambort Francisco, Castellano Matias.

El objetivo financiero en *PrestaMatch* consiste en colocar un presupuesto estricto de $2.000.000 USD maximizando el retorno neto. Métricas como *Accuracy* o *F1-Score* resultan inadecuadas debido a la severa asimetría en los costos de error: un acierto (aprobar a quien paga) genera únicamente la ganancia por tasa de interés ($+ Monto \times Tasa$), mientras que un Falso Positivo (aprobar a quien cae en default) destruye el 100% del capital prestado ($- Monto$). Rechazar solicitudes tiene costo cero. Por ello para medir el rendimiento de nuestros modelos, lo que tuvimos en cuenta fue lo siguiente: Monto total colocado, Cantidad de créditos aprobados, cantidad de default y ganancia.

Modelado y Preprocesamiento de Datos

Se integró la información tabular de solicitudes_train.csv con comportamiento_bancario.csv (661.139 registros mensuales de hasta 24 meses). Para evitar sobreajuste se descartaron las variables con cardinalidad alta: ID_Solicitud e ID_Usuario. Ademas descartamos Fecha_Solicitud, porque la creimos irrelevante.

Feature Engineering Financiero y Reducción Dimensional

La ingeniería de variables se guió por el impacto de los coeficientes estandarizados en modelos lineales.
Se analizó el historial bancario separando la gravedad de la mora (Dias_Atraso_Max) de la frecuencia o hábito (Dias_Atraso_Mean). Al observar los coeficientes lineales, notamos que Dias_Atraso_Max tenia un coeficiente positivo (contrario a lo esperado), por lo que decidimos conservar únicamente Dias_Atraso_Mean ($\beta = -1.08$).
Se generó Esfuerzo_Pago ($Monto/Ingreso$), de esta forma buscamos una relacion entre estas dos variables, para ver la capacidad de pago. ($\beta = -0.48$). Se diseñó Uso_Dinero ($Saldo_Promedio_6M/Ingreso$). Lo que se busco con esta variable, es ver que tan al límite viven nuestro clientes, porque de nada nos sirve que tenga un buen salario si se lo gasta todo.($\beta = +0.42$). Finalmente, las 7 categorías de Motivo_Prestamo obtenidas por One-Hot Encoding se compactaron en grupos de riesgo tras constatar coeficientes equivalentes: Alta Confianza (Educación, Vivienda, Salud; $\beta \approx +0.45$), Neutro (Consumo, Otros, Vehículo; $\beta \approx +0.20$) y Riesgoso (Negocio; $\beta \leq 0$)

Estrategia de Selección de Cartera y Asignación Presupuestaria

La forma en que se decide a quién entregarle los $2.000.000 USD de presupuesto resultó ser tan importante como el modelo mismo. En las pruebas evaluamos tres formas de armar la cartera:
Predicción Binaria y Orden por Monto (El inicio): Al principio nos quedamos con los clientes que el modelo predecía con un 1 (aprobar) y los ordenamos por monto de menor a mayor para diversificar. Si bien esto evitaba concentrar el riesgo en préstamos gigantes, no distinguía quién era más confiable dentro del grupo de aprobados y nos daba resultados modestos (35000 USD) al probar lo inverso los resultados fueron muy malos (-5000 USD) porque un par de morosos grandes se comían todo el dinero.

Valor Monetario Esperado (Retorno teórico): Probamos ordenar calculando la ganancia esperada (lo que podíamos ganar por intereses menos lo que podíamos perder si no pagaban). Aunque la fórmula tiene sentido en los papeles, en la práctica no fue la mejor opción: tendía a darle prioridad a préstamos con tasas altas o montos grandes donde el beneficio teórico parecía enorme, pero si el cliente fallaba, la pérdida de capital era devastadora.
Probabilidad de Pago (ganador)**:** El verdadero salto se dio cuando dejamos de usar la predicción rígida de 0 o 1 y empezamos a extraer la probabilidad exacta de pago con predict_proba. Al ordenar a todos los solicitantes de mayor a menor probabilidad, el banco le presta primero a los clientes más sólidos y seguros de todo el conjunto. Fue precisamente bajo este criterio de ranking por probabilidad donde logramos los mejores rendimientos del proyecto, demostrando que en crédito la prioridad número uno siempre debe ser blindarse contra el default.

Experimentación: Árboles y Boosting

Al principio, los modelos basados en árboles nos mostraron una ilusión peligrosa en las pruebas locales: Parecían perfectos, pero al subirlos a Kaggle el rendimiento real bajo a $146.079 USD en Gradient Boosting y $143.998 USD en Random Forest.

¿Por qué pasó esto?

Exceso de confianza en las probabilidades: Los árboles son capaces de crear reglas muy específicas y aislar grupos chicos de clientes. Si en el entrenamiento un grupo de personas pagó siempre, el árbol asume que cualquier persona parecida tiene casi 100% de probabilidad. En Kaggle nadie tiene riesgo cero: bastó que fallaran un puñado de esos clientes "perfectos" para que la pérdida de capital destruyera la ganancia.

Intento de regularización: Para evitar que el Gradient Boosting memorice tanto, usamos *GridSearch* para ponerle límites estrictos: le bajamos la profundidad a los árboles, le exigimos que cada regla agrupe al menos a 20 clientes y bajamos el ritmo de aprendizaje. Aunque esto mejoró la prudencia del modelo y empezó a reconocer riesgo real en local, en la colocación final de la cartera siguió rindiendo por debajo de la Regresión Logística.

Solución Final Propuesta y Justificación

Modelo Seleccionado: Regresión Logística, entrenada sobre el conjunto curado de variables financieras: Esfuerzo_Pago, Uso_Dinero, Dias_Atraso_Mean y las variables de motivo préstamo con one hot encoding. Esta version nos dio un leve mejor resultado que cuando compactamos los motivos de prestamo en en grupos de riesgo.
Rendimiento Financiero: 159243.04 USD
No memoriza de más: Los árboles crearon reglas rebuscadas que memorizaron los datos locales; la regresión logística trazó una tendencia general simple que no falló con clientes nuevos en Kaggle.
Sentido común y total control: Pudimos ver los coeficientes y comprobar que castiga a los morosos y sobreendeudados, y premia a quienes tienen colchón de ahorro. No es una caja negra: sabemos exactamente por qué aprueba a cada persona.